

Workshop 1: Entity-Relationship Diagram (ERD)

2026-03-25

Contents

1	Several recalls (10 min.)	1
2	Be positive! (5 min.)	2
3	Let's do it step by step! (5 min.)	2
4	Boats and Customers (10 min.)	2
5	Adding the Problems (10 min.)	2
6	Fika break! (15 min.)	3
7	Adding the Employees (10 min.)	3
8	Adding the Persons (10 min.)	3
9	From conceptual (ERD) to logical (tables) (15 min.)	4
10	If you have finished...	4

In this workshop, you will work in a team on a table with many other students around a boats/customers/problems/employees database. It is a collective discussion to find a solution to the given problem, step by step by exchange and comments. You need paper and pens or "draw.io" to draw the Entity-Relationship Diagram (ERD) and a spreadsheet (Excel?) to create the tables. Please be positive, let all the people speak around the table, do not speak too loud, accept the comments for others, even negative comments can be treated as a way to improve your solution (always be positive). You can ask questions to me at any moment during the session. Välkomna, welcome, bienvenue...

1 Several recalls (10 min.)

The Entity-Relationship Diagram (ERD) is a diagram (see example in Figure 1) describing a model of the database. It contains **entities (rectangle)**, having **attributes (ellipses)**. It contains **relationships (diamonds)** that link at least two entities with a verb. It also contains the **cardinalities (min:max)**, one per relation branch.

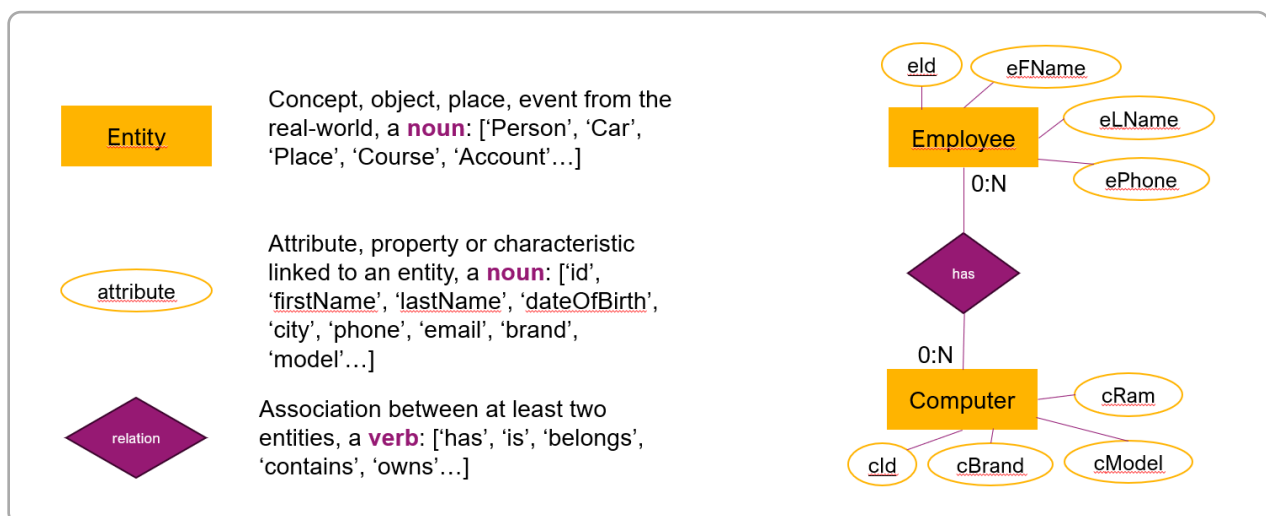


Figure 1: Entity-Relationship Diagram (ERD) components.

2 Be positive! (5 min.)

Because, perhaps, you don't know each other around the table, take one minute per person to present yourself, explain who you are, where you come from, why you are taking this course/program and what you want to do in your future professional life. If everyone around the table is speaking Swedish, you can do it in Swedish, else, please go back to English.

3 Let's do it step by step! (5 min.)

We have the following situation, please read the text:

—

A company renting boats needs a database to manage the boats, customers, employees and problems on the boats. A boat has a boatID, a name, a size (in meters), a capacity (maximum number of persons on board), the boat is attached to a port (a city name, for instance 'Stockholm', 'Göteborg', ...). Customers have a customerID, a first name, a last name, a date of birth, a phone number and an email address. Customers can rent a boat from a start date to an end date. The boats can be rent by many customers and a customer can rent many boats. The customers can declare a problem on a boat. A customer can declare multiple problems but a problem is declared by one customer only. A problem has a problemID, the boatID of the concerned boat, a date and a description (a text written by the customer to describe the problem on the boat). One or more employees will solve the problem at a certain date, with a timePassed (the number of hours they spent on this task) and the solution description. Employees and Customers are Persons. An Employee can be a Customer and a Customer can be an Employee.

—

Discuss with your colleagues around the table about **question 1**: how many entities do you need to represent this situation? We will see at the end of today's session if you were right.

Instead of considering the full situation, we will break it into smaller parts. We will build an Entity-Relationship Diagram for this database, step by step.

4 Boats and Customers (10 min.)

We now consider ONLY the text:

— A company renting boats needs a database to manage the boats, customers, employees and problems on the boats.

A boat has a boatID, a name, a size (in meters), a capacity (maximum number of persons on board), the boat is attached to a port (a city name, for instance 'Stockholm', 'Göteborg', ...).

Customers have a customerID, a first name, a last name, a date of birth, a phone number and an email address.

Customers can rent a boat from a start date to an end date. The boats can be rent by many customers and a customer can rent many boats.

—

With your colleagues around the table, **discuss and draw** a full ERD (entities, attributes, relationships and cardinalities) to represent this situation.

5 Adding the Problems (10 min.)

We add another part of the text:

The customers can declare a problem on a boat. A customer can declare multiple problems but a problem is declared by one customer only. A problem has a problemID, the boatID of the concerned boat, a date and a description (a text written by the customer to describe the problem on the boat).

With your colleagues around the table, **discuss and update your design** to represent the Boats, Customers and Problems.

6 Fika break! (15 min.)

It is time to take a 15 minute pause, go outside, walk, take a coffee, relax.

7 Adding the Employees (10 min.)

We add another part of the text:

One or more employees will solve the problem at a certain date, with a timePassed (the number of hours they spent on this task) and the solution description.

With your colleagues around the table, **discuss and update your design** to represent the Boats, Customers, Problems and Employees.

8 Adding the Persons (10 min.)

We add another part of the text:

Employees and Customers are Persons. An Employee can be a Customer and a Customer can be an Employee.

With your colleagues around the table, **discuss and update your design** to represent the Boats, Customers, Problems and Employees knowing that Employees and Customers are Persons.

Because Employees and Customers are Persons and Employees can be Customers and Customers can be Employees, you will need to use the concept of inheritance (see Figure 2) in your final ERD.

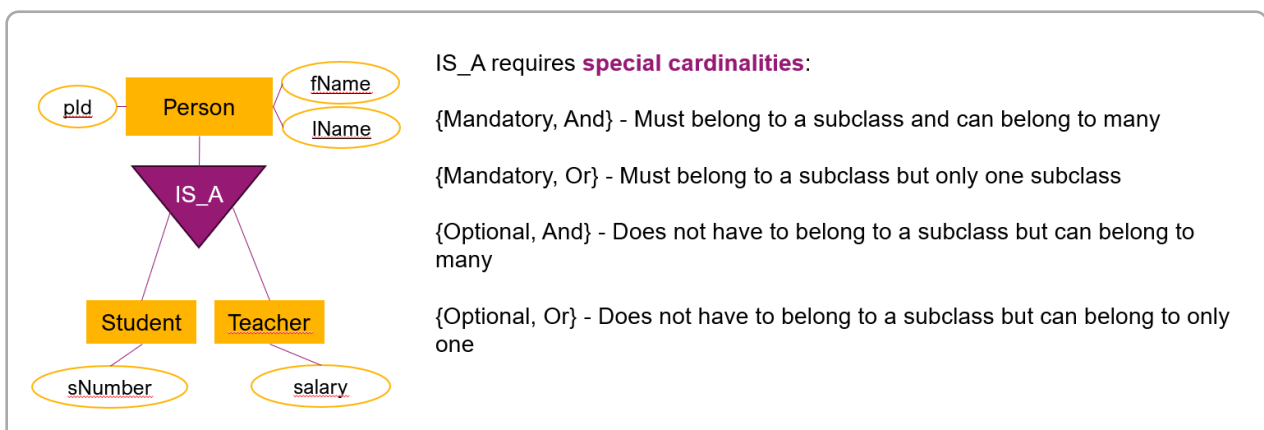


Figure 2: The concept of inheritance.

9 From conceptual (ERD) to logical (tables) (15 min.)

- Following the rules given in Figure 3, with your colleagues around the table, **discuss and update your design** to represent the logical model (tables) in your database.

RULES:

- 1) Any **entity** becomes a **table**.
- 2) **Look at the cardinalities (max only):**
 - a. a **one-to-many** cardinality makes the **PK** on the 1 side become a **FK** into the table on the * side.
 - b. a **many-to-many** relation becomes a **full table** with **PK** being the two **FK** of the tables in the relation.

Figure 3: The rules to follow to go from the **conceptual** to the **logical** model.

Of course, the special inheritance "IS_A" is a little bit tricky to transform because it is not included in the rules. However, you can guess with your colleagues how you would transform it into tables.

10 If you have finished...

In case you have finished the workshop very fast, you can work as a group on the lab assignment. I know you will also work with your partner on it, but it can help to start drawing some parts of the ERD by discussing with your colleagues around the table. :)